ELECTRÓNICA DIGITAL I · SEGUNDA ENTREGA · CLASE 3 DE 5

Contadores y base de tiempo

Un solo reloj de 50 MHz y muchos tiempos distintos: 1 ms, 20 ms, 200 ms, 1 s, 15 s, 30 s.

PARA LA FPGA EL TIEMPO SE CUENTA

50 000 000

flancos de reloj caben en un segundo en la DE0-Nano (50 MHz, periodo de 20 ns)

Antirrebote · 20 ms

1 000 000 ciclos

Watchdog · 15 s

750 000 000 ciclos

Bloqueo · 30 s

1 500 000 000 ciclos

Cuenta de 0 a N–1 y vuelve a empezar

```
signal cuenta : integer
  range 0 to N - 1 := 0;

if rising_edge(clk) then
  if cuenta = N - 1 then
    cuenta <= 0;
  else
    cuenta <= cuenta + 1;
  end if;
end if;
```

Permitido

+ – = < > sobre señales. CLK_HZ / 1000 entre constantes: Quartus lo calcula al compilar.

Prohibido (R3, R7)

/ mod rem * sobre señales. Las librerías std_logic_arith y std_logic_unsigned.

UN SOLO RELOJ · R1

No se fabrican relojes: se fabrican habilitaciones

MAL · reloj derivado

```
if cuenta = N - 1 then
  clk_lento <= not clk_lento;
end if;

process (clk_lento)
  if rising_edge(clk_lento)
    q <= not q;
```

clk_lento sale de un flip-flop y llega tarde a cada destino. Quartus no puede garantizar los tiempos.

BIEN · tick + clock enable

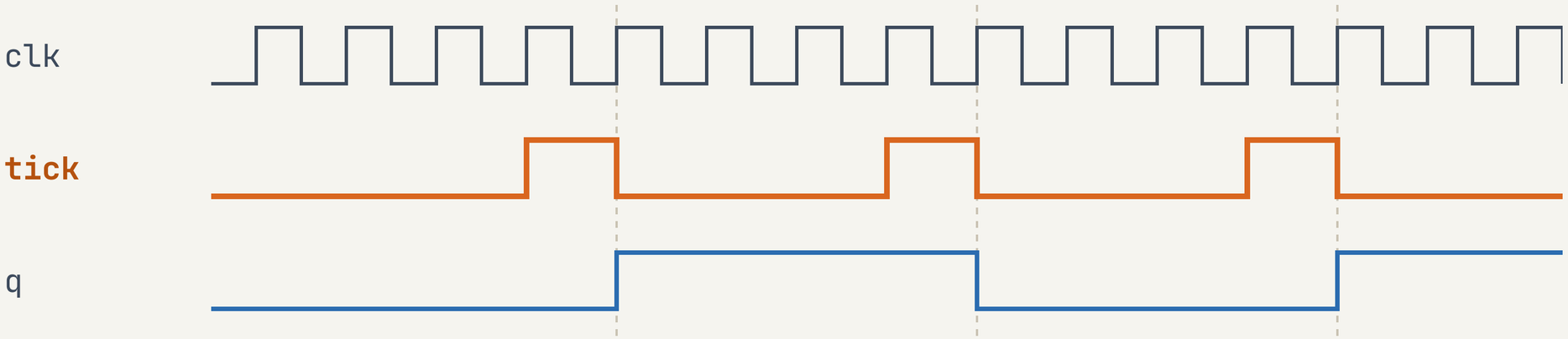
```
tick <= '0';
if cuenta = N - 1 then
  tick <= '1';
end if;

if rising_edge(clk) then
  if tick = '1' then q <= not q;
```

Todos los flip-flops ven el mismo flanco. tick dura un ciclo y solo dice cuándo actuar.

DIAGRAMA DE TIEMPOS · EJEMPLO CON N = 4

El tick dura exactamente un ciclo de reloj

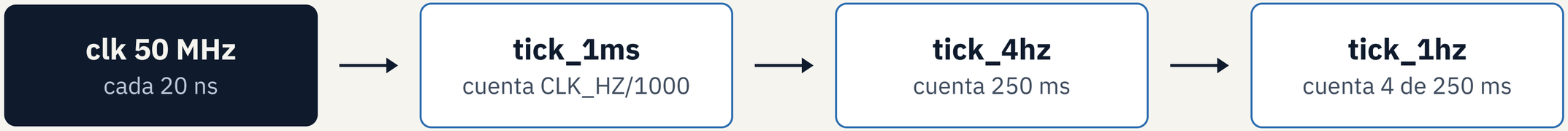


El contador levanta tick cuando llega a N – 1. En el siguiente flanco, quien lo use ve tick = 1 y actúa.

Si tick durara dos ciclos, q cambiaría dos veces seguidas. Por eso cada proceso arranca con **tick <= '0'**.

BASE_TIEMPO.VHD

Tres contadores en cascada dan todos los tiempos



Tick	Quién lo usa
tick_1ms	barrido del teclado · antirrebote 20 ms · filtro de oleaje 200 ms · error de sensor 1 s
tick_4hz	mensajes de 2 s · onda_2hz para el parpadeo de alarma (flip-flop T)
tick_1hz	bloqueo 30 s · inactividad 20 s · watchdog 15 s

GENERICS · R10

El mismo código corre a 50 MHz y a 10 kHz

En el diseño

```
entity base_tiempo is
  generic (
    CLK_HZ : positive
      := 50_000_000);
...
constant CICLOS_MS : positive
  := CLK_HZ / 1000;
```

En el testbench

```
u_bt : entity work.base_tiempo
  generic map (
    CLK_HZ => 10_000)
  port map (...);
```

Tarjeta: **50 000 ciclos** por milisegundo.

Simulación: **10 ciclos** por milisegundo. Los 30 s del bloqueo se simulan en segundos.

TEMPORIZADOR.VHD

Un temporizador genérico sirve para tres requerimientos

Uso	tick	N_TICKS	ena	reiniciar
Inactividad · RF7	tick_1hz	20	sesión abierta	tecla_valida
Error de sensor · RF9	tick_1ms	1000	lectura inválida	'0'
Mensaje Err · 2 s	tick_4hz	8	mensaje activo	'0'

ena = 1

Cuenta un paso por cada tick.

Llega a N

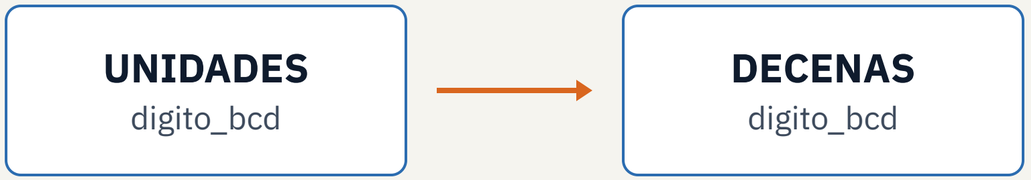
fin = 1 y se queda en 1.

ena = 0 o reiniciar

Todo vuelve a cero.

TEMPORIZADOR_BCD.VHD · R7

Dos dígitos BCD en cascada, sin dividir nunca



Las unidades bajan con cada tick. Si estaban en 0 pasan a 9 y levantan **presta**: ese pulso hace bajar las decenas.

En el proyecto: la cuenta regresiva de 30 s del bloqueo (RF2), directa a los displays.

antes	después	presta
12	11	0
11	10	0
10	**09**	**1**
09	08	0
01	00	0
00	00 · cero	0

TALLER EN CLASE Y TAREA

Meta: tb_clase3 con 16 OK

1 · base_tiempo

Etapas 2 y 3, anidadas en la etapa 1.

2 · temporizador

El process completo, con la tabla de la diapositiva 8.

3 · digito_bcd

De 0 a 9, y las ecuaciones de cero y presta.

4 · temporizador_bcd

dec_u, cero_tot y la instancia de las decenas.

Tarea · watchdog_bomba

RF8: falla tras T_S segundos de bomba sin que suba el nivel.

Con la plantilla sin tocar verán 6 OK y 10 FALLAS. Los 3 últimos casos son de la tarea.